WHY CONTEXT MATTERS

How 1 million tokens changes everything for brands, builders, and teams.

A PRACTICAL GUIDE TO BUILDING WITH CLAUDE

Brand profiles. Context swipe files. Agent teams.
Stop giving AI guesswork. Start giving it context.

MARCH 2026

Henry Nicholson

linkedin.com/in/henrymnicholson

CONTENTS

01

THE CONTEXT PROBLEM

Every time you open a new chat with an AI model, you start from zero. The model doesn't know your brand, your clients, your tech stack, your tone, or your goals. It doesn't know that your agency avoids exclamation marks in B2B copy, or that your React projects always use Tailwind, or that your client's brand voice is 'confident but never cocky.'

So what happens? You get generic output. You get copy that sounds like it was written by a committee of strangers. You get code scaffolded with the wrong packages. You waste 15 minutes re-explaining things that should already be understood.

> *This is the context problem. And it is the single biggest reason most people get mediocre results from AI.*

The fix isn't better prompting tricks. It's not a magic system prompt. It's giving the model the same context you'd give a new team member on day one: who you are, what you do, how you do it, and what good looks like.

Context is what turns a general-purpose language model into YOUR language model.

02

WHAT 1M TOKENS ACTUALLY MEANS FOR BRANDS

On February 5, 2026, Anthropic released Claude Opus 4.6 with a 1 million token context window. Two weeks later, Sonnet 4.6 followed with the same capacity. This isn't an incremental upgrade. It's a structural shift in what's possible.

BY THE NUMBERS

- 1 million tokens = roughly 750,000 words
- That's approximately 1,500 pages of text in a single prompt
- Or 30,000+ lines of code with full comprehension
- Or over an hour of video transcript

WHY THIS MATTERS FOR BRANDS AND AGENCIES

Before this, you had to choose what context to include. With 200K tokens, you could fit a brand guide OR a batch of reference material, but rarely both. You were constantly summarizing, trimming, and hoping the model could fill in the gaps.

With 1M tokens, you can load your entire brand ecosystem into a single conversation: brand guidelines, tone documentation, past campaign examples, client briefs, competitive research, and style references. All at once. And the model actually uses it. Opus 4.6 scores 76% on MRCR v2 (needle-in-a-haystack retrieval across 1M tokens) compared to just 18.5% for the previous Sonnet 4.5. That's a 4x improvement in how reliably the model finds and applies information buried deep in your context.

> *This means the era of 'close enough' AI output is over. If your context is good, your output will be good.*

CONTEXT COMPACTION

Opus 4.6 also introduces context compaction: when conversations approach the window limit, the model automatically summarizes earlier content to keep going. In theory, this means infinite conversation length without losing the thread. For long-running projects, client work, and iterative creative briefs, this is a game changer.

03

BUILDING BRAND PROFILES & CONTEXT SWIPE FILES

A context swipe file is a reusable document (in .txt or .md format) that captures everything Claude needs to know about a brand, client, project, or person. You create it once, reference it every time, and the result is consistent, on-brand output across every conversation.

WHAT GOES INTO A CONTEXT SWIPE FILE

- **Identity:** Who this is. Company name, mission, positioning, what makes them different.
- **Voice & Tone:** How they sound. Formal or casual? First person or third? Do they use contractions? What words do they never use?
- **Style Rules:** Formatting preferences, capitalization, preferred structures, emoji usage.
- **Do Say / Don't Say:** Phrases that are on-brand vs. phrases that are off-limits.
- **Audience:** Who they're talking to. Demographics, psychographics, pain points.
- **Examples:** 2-3 samples of past content that nails the tone. This is the most powerful element.
- **Technical Stack:** (For dev work) Languages, frameworks, packages, conventions, file structure.

HOW TO USE THEM

Save your context file as a .md or .txt. At the start of any Claude conversation, upload the file or paste it in. Every response in that thread will now be informed by the full context. For Claude Projects, you can pin context files so they're automatically included in every conversation within that project. For API users, include the file in your system prompt.

> *Think of it as onboarding a new team member, except this team member has perfect recall and never forgets a single detail from the brief.*

04

COPY-PASTE PROMPT: BRAND VOICE PROFILE

Send this prompt directly to Claude to generate a comprehensive brand voice profile. Save the output as a .md file and reuse it across all future conversations.

PROMPT: BRAND VOICE GENERATOR

```
I need you to help me build a comprehensive brand voice profile that I can reuse as a context file in future conversations. Ask me the following questions one at a time, then compile the answers into a structured .md file: 1. What is the brand/company name and what do they do? 2. Who is the target audience? (demographics, mindset, pain points) 3. Describe the brand personality in 3-5 adjectives. 4. What tone should copy have? (formal/casual/playful/ authoritative/etc.) 5. Provide 2-3 examples of content that perfectly captures the brand voice. 6. What words, phrases, or styles should be AVOIDED? 7. What words, phrases, or styles should be USED often? 8. Any formatting preferences? (sentence case, no emojis, Oxford comma, etc.) 9. What is the brand's core message or tagline? 10. Who are the main competitors and how should this brand differ from them in voice? After gathering all answers, output a clean markdown file titled [BRAND]_VOICE_PROFILE.md with sections for: - Brand Overview - Target Audience - Voice Attributes - Tone Guidelines - Do Say / Don't Say - Style Rules - Example Content - Competitive Positioning Make the file ready to be uploaded to any future Claude conversation as a context reference.
```

05

COPY-PASTE PROMPT: TECHNICAL / DEV PROFILE

For developers, vibe coders, and builders. This prompt generates a technical context file that eliminates the guesswork Claude normally has around your tech stack, UI preferences, and coding patterns.

PROMPT: TECHNICAL PROFILE GENERATOR

```
Help me create a technical context profile that I can reference in every coding conversation. Ask me these questions one at a time, then compile into a .md file: 1. What languages do you primarily code in? 2. What frameworks do you use? (React, Next.js, Flask, Express, etc.) 3. What CSS approach? (Tailwind, CSS Modules, styled-components, vanilla) 4. What package manager? (npm, yarn, pnpm, pip) 5. What database(s) do you typically use? 6. Preferred file/folder structure conventions? 7. Do you use TypeScript or JavaScript? Any lint rules? 8. What deployment platform? (Vercel, AWS, Railway, etc.) 9. UI/UX preferences? (minimal, glassmorphism, dark mode, specific design systems) 10. Any libraries you always include? (framer-motion, zustand, shadcn/ui, etc.) 11. Code style preferences? (functional vs class components, naming conventions, comment style) 12. Any patterns to AVOID in your projects? Output a clean markdown file titled [NAME]_TECH_PROFILE.md with sections for: - Tech Stack Overview - Framework & Library Preferences - Code Style & Conventions - UI/UX Guidelines - Project Structure Template - Deployment & Infrastructure - Do / Don't Patterns This file should be reusable as context in any future Claude conversation or Claude Code session.
```

06

COPY-PASTE PROMPT: CLIENT CONTEXT FILE

For agencies and freelancers managing multiple clients. Generate a dedicated context file per client so every piece of work, from social posts to email campaigns to landing pages, stays consistent.

PROMPT: CLIENT CONTEXT GENERATOR

```
I manage work for multiple clients and need a dedicated context file for one of them. Interview me with these questions, then compile into a reusable .md file: 1. Client name and industry? 2. What services do you provide for this client? 3. Describe their brand voice in detail. How do they want to sound to their audience? 4. Who is their target audience? 5. What channels do you create content for? (social, email, web, print, etc.) 6. Share 2-3 examples of approved content that represents their voice well. 7. Any restricted topics, words, or styles? 8. Key products, services, or offerings to reference? 9. Brand colors, fonts, or visual style notes? 10. Competitor brands to be aware of? 11. Current campaign themes or messaging priorities? 12. Any internal terminology or jargon specific to this client? Output as [CLIENT_NAME]_CONTEXT.md with sections for: - Client Overview - Brand Voice & Tone - Target Audience - Content Channels & Formats - Approved Examples - Restrictions & Guardrails - Key Offerings - Campaign Priorities - Visual Identity Notes Make it ready to drop into any Claude conversation or Claude Project as persistent context.
```

07

THE WRITER WHO CAN'T WRITE (AND DOESN'T NEED TO)

> *Here's a truth nobody talks about: you don't need to become a better writer. You need to become a better briefer.*

If writing isn't your strength, the worst thing you can do is spend months trying to level up your prose while deadlines pile up. Instead, invest that time once into building a personal voice profile that Claude will follow every time you need copy.

WHAT A PERSONAL VOICE PROFILE DOES

- Captures how you naturally communicate (even if you can't articulate it yourself)
- Defines your preferred sentence length, vocabulary level, and personality
- Includes examples of writing you like, so Claude can pattern-match
- Sets guardrails so output never sounds generic or off-brand

THE PROCESS

1. Gather 3-5 pieces of writing you've done (or that represent how you want to sound). Emails, social posts, Slack messages, anything authentic.

2. Send them to Claude with the brand voice generator prompt from Section 04.

3. Claude analyzes the patterns: your sentence structure, word choice, rhythm, and personality.

4. Save the output as YOUR_NAME_VOICE.md. Drop it into every conversation where you need copy.

The result: every email, social caption, blog post, and pitch deck sounds like you wrote it, because you defined what 'you' sounds like. You're not outsourcing your voice. You're scaling it.

PROMPT: PERSONAL VOICE ANALYZER

```
Analyze the following writing samples and create a personal voice profile for me. Identify: - Average sentence length and structure patterns - Vocabulary level (casual, professional, technical) - Personality traits that come through - Recurring phrases or stylistic habits - Tone (confident, conversational, formal, etc.) - What makes my writing distinct from generic copy Then compile this into a reusable VOICE_PROFILE.md that I can reference in future conversations so all output matches my natural communication style. [PASTE YOUR WRITING SAMPLES BELOW]
```

08

THE DEVELOPER WHO SKIPS THE SPEC

When you ask Claude to 'build me a dashboard' without context, it has to guess everything: React or Vue? Tailwind or CSS Modules? REST or GraphQL? Light mode or dark? And every guess is a coin flip that might not match your project.

A technical context file eliminates the guesswork. It's a one-time investment that saves hours of back-and-forth on every project.

WHAT CHANGES WITH CONTEXT

WITHOUT CONTEXT:

```
User: "Build me a login page" Claude: *generates vanilla HTML/CSS/JS with Bootstrap* User: "No, I use React and Tailwind" Claude: *rebuilds with React + Tailwind* User: "I also use shadcn/ui components" Claude: *rebuilds again* // 3 attempts. 15 minutes wasted.
```

WITH CONTEXT:

```
User: [tech_profile.md uploaded] User: "Build me a login page" Claude: *generates Next.js + Tailwind + shadcn/ui with your preferred dark theme, your folder structure, and your auth patterns* // First attempt. Done.
```

FOR CLAUDE CODE USERS

If you're using Claude Code, create a CLAUDE.md file at the root of your project. This file is automatically read at the start of every session. Put your tech stack, conventions, and project-specific context in here. Every agent, subagent, and teammate will inherit this context.

PROMPT: CLAUDE.MD GENERATOR

```
Based on my tech profile, generate a CLAUDE.md file for my project root that includes: - Project overview and purpose - Tech stack with versions - File structure conventions - Code style rules (naming, patterns, imports) - UI/UX guidelines and component library usage - Testing approach - Common commands (dev, build, deploy) - Do / Don't rules for this codebase Format it so Claude Code reads it on startup and follows all conventions automatically. [PASTE YOUR TECH PROFILE OR DESCRIBE YOUR PROJECT]
```

09

AGENT TEAMS: PARALLEL WORK, CONSISTENT OUTPUT

With the release of Opus 4.6, Anthropic introduced Agent Teams in Claude Code. This is a research preview feature that lets you spawn multiple Claude instances that work in parallel, communicate with each other, and coordinate through a shared task list.

> *Think of it as going from one developer to an entire engineering team, all running simultaneously.*

HOW IT WORKS

- **Lead Agent:** Your main Claude Code session. Creates the team, assigns tasks, synthesizes results.
- **Teammates:** Separate Claude instances, each with their own context window. They load project context (CLAUDE.md, MCP servers) but work independently.
- **Shared Task List:** Central work items that move through pending, in progress, and completed states.
- **Direct Messaging:** Teammates can message each other. If the backend agent discovers something the frontend agent needs, it shares it directly.

AGENT TEAMS VS. SUBAGENTS

Subagents are quick, focused workers that report back to you. They can't talk to each other. Agent Teams are full collaborators: they share findings, challenge each other's conclusions, and self-coordinate. Use subagents for simple delegation. Use agent teams when the work requires real collaboration across multiple domains.

BEST USE CASES

- Research and review: multiple agents investigate different aspects simultaneously
- New features: one agent per module, all building in parallel
- Debugging: competing hypotheses tested simultaneously, agents challenge each other
- Cross-layer work: frontend, backend, and tests each owned by a different teammate
- Code review: security, performance, and maintainability reviewed in parallel

10

HOW TO GIVE AGENT TEAMS STARTING CONTEXT

This is where everything in this guide converges. Agent teams are powerful, but without proper starting context, you get parallel chaos instead of parallel productivity. Each teammate gets its own context window. They don't inherit the lead's conversation history. The only shared knowledge comes from project files and the messages they exchange.

> *The context you provide at spawn time is everything. It determines whether your agents produce cohesive, on-brand work or fragmented output that doesn't connect.*

THE CONTEXT STACK FOR AGENT TEAMS

Layer 1: CLAUDE.md (Project-Level)
Every teammate reads this on startup. Put your tech stack, coding conventions, and project rules here. This is your universal baseline.

Layer 2: Context Swipe Files (Brand/Client-Level)
Reference your brand voice profile, client context file, or design system docs. These ensure all agents produce output that sounds and looks like it came from the same team.

Layer 3: Spawn Prompt (Task-Level)
When the lead spawns each teammate, the spawn prompt defines their specific role, scope, and constraints. Be explicit about what files they own, what patterns to follow, and what to avoid.

ENABLE AGENT TEAMS

```
# In your shell or settings.json: export CLAUDE_CODE_EXPERIMENTAL_AGENT_TEAMS=1 # Or in settings.json: { "env": { "CLAUDE_CODE_EXPERIMENTAL_AGENT_TEAMS": "1" } }
```

EXAMPLE: SPAWNING A COORDINATED TEAM

```
Create an agent team to build the new dashboard feature. Reference the project CLAUDE.md for all conventions. Spawn 3 teammates: 1. Backend Agent: Owns /src/server/. Build the API endpoints. Follow our Express middleware patterns. Use TypeORM for database operations. 2. Frontend Agent: Owns /src/client/. Build the React components. Use our shared component library in /src/client/components/shared/. Follow Tailwind conventions. 3. Test Agent: Owns /tests/. Write tests as each feature lands. Use Jest with our custom test utilities in /tests/helpers/. Coordinate through the shared task list. Frontend should wait for API types from Backend before building data fetching. Test agent should validate each piece as it completes.
```

Notice the pattern: each agent gets a clear scope (which files they own), specific conventions to follow, and explicit dependency ordering. This is what turns parallel execution into coordinated engineering.

REAL-WORLD WORKFLOW: THE AGENCY MODEL

Here's how this all comes together for an agency or freelancer managing multiple clients.

STEP 1: BUILD YOUR CONTEXT LIBRARY

- One brand voice profile per client (using Section 06 prompt)
- One personal voice profile for your own brand (using Section 04 prompt)
- One tech profile if you do development work (using Section 05 prompt)
- Store all .md files in a dedicated /context folder

STEP 2: CREATE CLAUDE PROJECTS PER CLIENT

In Claude, create a Project for each client. Pin their context file as persistent context. Now every conversation within that project automatically inherits the client's brand voice, audience, and guidelines. No re-explaining. No drift.

STEP 3: WORK ACROSS CHANNELS CONSISTENTLY

Need a social caption? Open the client's project, describe the post. Need an email sequence? Same project, same context. Need to brief a junior team member? Share the context file. The voice stays locked no matter who's prompting or what channel the content is for.

STEP 4: ITERATE AND UPDATE

Context files aren't static. When a client's messaging evolves, when you learn what copy performs best, when new products launch, update the file. The next conversation automatically reflects the changes. Your AI becomes smarter about each client over time because YOU make the context smarter.

PROMPT: CONTEXT FILE UPDATER

```
Here is my current context file for [CLIENT NAME]: [PASTE EXISTING CONTEXT FILE] Update it with the following new information: - [new campaign theme, product launch, voice shift, audience insight, etc.] Maintain the existing structure and format. Merge new information naturally. Flag any contradictions with the existing profile so I can resolve them. Output the complete updated .md file.
```

12

START BUILDING TODAY

You don't need to implement everything in this guide at once. Start with the one thing that will have the highest impact on your work right now.

IF YOU'RE A MARKETER OR COPYWRITER:

Build one brand voice profile for your most important client. Use it for a week. Watch how much faster and more consistent your output becomes.

IF YOU'RE A DEVELOPER:

Create a CLAUDE.md for your current project. Include your stack, conventions, and patterns. Notice how Claude stops guessing and starts building exactly what you need.

IF YOU'RE AN AGENCY:

Set up Claude Projects for your top 3 clients with pinned context files. Train your team to use them. Measure the time saved.

IF YOU'RE BUILDING WITH CLAUDE CODE:

Enable agent teams. Start with a code review task to see how parallel agents coordinate. Then scale to feature work. Give each agent clear context and scope.

> *Context is the difference between AI that produces generic content and AI that produces YOUR content. Between code that sort of works and code that fits your architecture. Between parallel agents that produce fragmented output and a coordinated team that ships.*

Stop giving AI the guesswork. Start giving it context.

If you made it this far, comment "context" on the post. I want to know who actually reads these.

WHY CONTEXT MATTERS · MARCH 2026

Henry Nicholson

Co-Founder, ForeFront USD

linkedin.com/in/henrymnicholson